When Structured State Replaces Interaction History: An Empirical Study of Coding Agents

Alex Markov-Buturskiy

Abstract

Replacing an agent's interaction history with structured state can reduce prompt size, but the replacement must preserve information needed for later decisions. We evaluate this trade-off for a terminal-based coding agent, comparing model-written state, hybrid state with runtime-derived records, and state augmented with recent history. Additional comparisons vary the state-update protocol and schema. With model-written state alone, the solved rate falls from 61.3% for full history to 34.9%; retaining recent history raises it to 59.0%. The corresponding hybrid policy reaches 48.3%, against its own full-history reference of 51.7%. Neither the tested inline update protocol nor a task-adapted schema closes that gap. Smaller prompts also do not imply cheaper successful execution: on jointly successful executions, model-written state alone has a median peak-prompt ratio of 0.47 and a cost ratio of 2.57 relative to full history. These results characterize a capability–resource trade-off in the evaluated implementations. They support retaining recent interaction records within those systems, while leaving the incremental benefit of state over an equally sized history window unresolved.

1 Introduction

A coding agent learns about a task while trying to solve it. A command reveals a dependency, a test invalidates a proposed fix, or an unsuccessful edit rules out an approach. Subsequent decisions depend on which of these results remain available. Choosing what to keep in context therefore changes both what the agent can use and the resources needed to present it.

An append-only transcript preserves earlier messages without requiring the agent to decide immediately which details will matter later. Its disadvantage is growth: more of the record must be presented at each step. Structured state makes the opposite choice. Selected facts and pending work are retained in mutable fields, while the original messages can be discarded. This can remove redundancy, but it asks the state writer to anticipate which details will matter later.

SKILL.state proposes such a runtime architecture in place of conversational history [Badhe et al., 2026]. We study its transfer to a terminal-based coding agent. We ask whether state can replace history without reducing task success, and what resource savings remain over a complete task. We test context policies with a different model, agent, and benchmark. This does not reproduce the original study.

Three design choices motivate the comparisons. Keeping recent history preserves records without rewriting them. State ownership determines whether the model must record an event or the runtime can preserve it directly. Finally, the update protocol and schema determine how the model expresses changes to its memory. A failure under one set of choices does not identify which choice was responsible.

We compare full history with model-written state and with hybrid state combining model-written content and runtime-derived records. Within these families, we vary how much recent history is kept; within the hybrid family, we also vary failure records, update protocol, and schema. Success and resource use are evaluated together. Peak prompt size measures a different property from cumulative output or cost per observed success, and a memory policy can improve one while worsening another.

Restoring recent history improves the observed solved rate in both state-based families. The tested protocol and schema changes do not recover the full-history result. Resource comparisons show why a compact prompt alone cannot tell us whether an agent solves tasks efficiently. Retained records remain useful in these state-based agents, but the tests do not establish that state adds value over the same amount of history alone.

2 Context policies and research questions

History and state differ in what they preserve. History retains records in their original order. State rewrites selected details into fields that describe the current task. A state field may record that a test failed without preserving its complete output or the edits preceding it. Conversely, keeping those edits does not guarantee that a model will use it effectively.

There are also different ways to produce state. A model can maintain semantic content such as hypotheses and pending work. A runtime can record facts directly observable as the agent works, such as completed commands and their outcomes. We call a state that combines these sources *hybrid state*. Runtime-derived records reduce the need to restate observable events, but do not by themselves explain what those events mean or what the agent should do next.

The published SKILL.state contract jointly emits a state patch and an action [Badhe et al., 2026]. A separate state-update tool changes that interface: the agent updates memory through a tool call within its main loop. Our inline policy emits the patch and action together, while the schema comparison tests how the patch is organized. These changes test specific aspects of the port, not every way in which it differs from the published system.

We ask how keeping recent history affects success, whether hybrid state and failure records preserve useful details, and how protocol and schema choices change the outcome. We also ask whether smaller prompts lead to lower cost and more successes per dollar spent. Each comparison requires within-family controls. In particular, adding recent history to state tests the value of that history within a state-based agent; it does not isolate the value of the state block itself.

3 Context Policies and Update Protocols

Let P denote persistent task instructions, o_t the current observation, and $H_{<t}$ the retained interaction records preceding it. The record includes observable messages and tool results; the notation makes no assumption about access to hidden model reasoning. A full-history policy constructs

$$C_t^{\text{full}} = [P, H_{<t}, o_t]. \tag{1}$$

For structured state Σ_t, the replacement policy is

$$C_t^{\text{state},k} = [P, \Sigma_t, \tau_k(H_{<t}), o_t], \qquad \tau_0(H) = \varnothing, \tag{2}$$

where τ_k selects the configured recent-history window, clipped at the start of the task. Full history is a separate reference policy, not state with an unlimited window. Figure 1 shows the information retained by these policies.

3.1 Who writes the state?

In model-written memory, the model maintains semantic state S_t through explicit updates. Hybrid memory separates this content from a runtime record:

$$\Sigma_t = (S_t, V_t), \qquad S_{t+1} = \text{Apply}(S_t, \Delta S_t), \qquad V_{t+1} = \text{Record}(V_t, a_t, r_t). \tag{3}$$

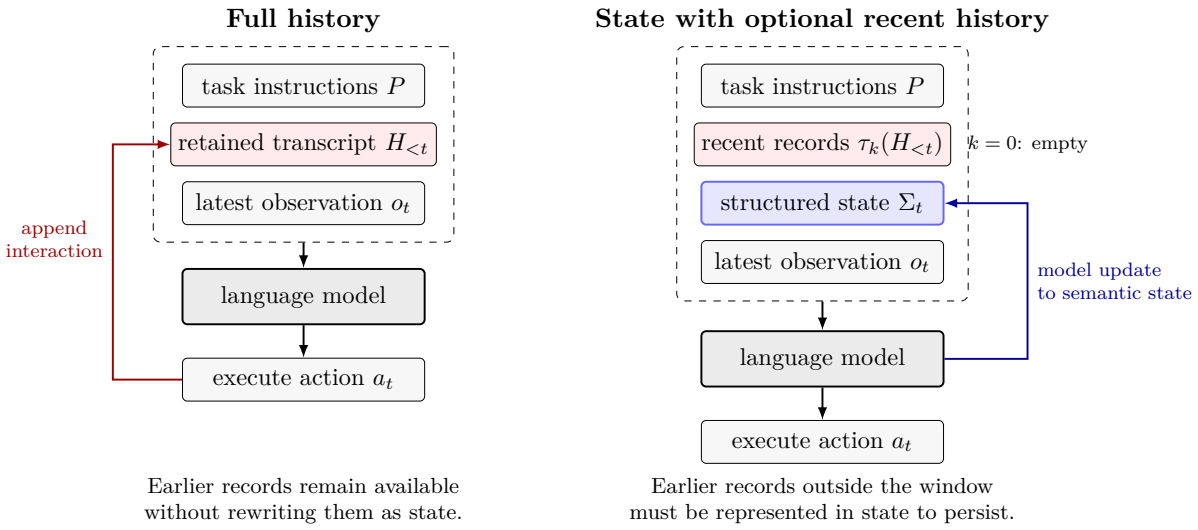


Figure 1: Context policies. State replaces the older transcript, while the retention parameter k controls the recent suffix. The arrows show information flow, not the number of model calls.

The recorder derives entries from observed tool events r_t; the model cannot edit those entries through its semantic patch. This ownership distinction does not guarantee complete capture or correct interpretation of every event. The evaluated hybrid variants differ in whether failed-attempt records are included. Recent history is an independent addition to either state family.

3.2 How is an update coupled to an action?

A separate update tool lets the model submit a state patch through the tool interface. An inline protocol instead asks for a joint envelope containing $\{\Delta S_t, a_t\}$, following the form of the published SKILL.state contract [Badhe et al., 2026]. These protocols differ in the relationship between state maintenance and environment actions; a tool-mediated policy step need not correspond to a single model invocation.

Algorithm 1 specifies the context and ownership abstraction used to distinguish the policies. Its PolicyStep leaves message serialization and the number of tool-mediated calls unspecified; with full history it emits an action and leaves S unchanged. Algorithm 2 describes the joint patch–action contract, including rejection before action execution. It is a reference contract for the comparison, not a recovered execution trace or a claim that every evaluated implementation obeyed every detail. A fixed retry count or automatic fallback to full history is not assumed.

Algorithm 1 Context selection and state ownership

Require: Instructions P, policy π, retention setting k, update protocol q, initial observation o

Initialize history H, semantic state S, runtime record V
while the task has not terminated **do**
  if $\pi =$ FullHistory **then**
    $C \leftarrow [P, H, o]$
  else
    $\Sigma \leftarrow S$ **or** (S, V) according to state ownership
    $C \leftarrow [P, \Sigma, \tau_k(H), o]$
  end if
  $(S', a) \leftarrow \text{PolicyStep}(C, S, \pi, q)$
  $r \leftarrow \text{Execute}(a)$; $S \leftarrow S'$
  if hybrid memory is enabled **then**
    $V \leftarrow \text{Record}(V, a, r)$
  end if
  Append a record of (o, a) to H; $o \leftarrow r$
end while

Algorithm 2 Inline patch–action contract

Require: Current context C and semantic state S

$(\Delta S, a) \leftarrow \text{Model}(C)$
if $\text{Valid}(\Delta S, S)$ **then**
  return $(\text{Apply}(S, \Delta S), a)$
else
  Keep S unchanged and return a validation observation
  Do not execute a from the rejected envelope
end if

3.3 Context size and complete-task cost

If interaction records have bounded size and the number of model calls per action is bounded, retaining the full transcript gives prompt growth of order t and cumulative input growth of order T^2 over T actions. A state policy has an action-independent context bound only when the state, observation, and retained-history records are all bounded. Under those assumptions and fixed k, cumulative input grows as $O(T)$. Neither statement fixes how many actions the policy will take to finish.

Let U, K, and O be accumulated uncached-input, cache-read, and output tokens. With their respective tariffs α, γ, and β, normalized cost is

$$D = \alpha U + \gamma K + \beta O. \tag{4}$$

A smaller maximum prompt does not determine any of these task-level sums. State updates can add output, and a changing prompt can alter cache reuse; these are possible explanations to examine, not consequences proved by the context-size bound. The evaluation therefore reports success, prompt size, token categories, and cost separately.

4 Experimental methodology

4.1 Agent, tasks, and comparisons

The evaluation uses `zai/glm-5.3-flash`, Pi coding agent version 0.85.1, and the Harbor 0.22 release family. Tasks come from Terminal-Bench 2.0, which supplies executable environments

and task-specific verification tests [Merrill et al., 2026]. The agent works through coding and terminal tools. Success is the binary task-verifier outcome, and the recorded execution limit is 200 turns.

The analyzed subset contains 87 tasks; two further benchmark tasks were excluded because their environments could not be initialized before agent execution. The model-written comparison has four conditions with three repetitions per task. The hybrid comparison has eight conditions with two repetitions per task. Each comparison includes its own full-history control. An observation is a task execution under a condition and repetition, and contrasts match the same task and repetition. We do not pool the controls or treat repeated executions as additional tasks.

The model-written conditions use a separate state-update tool and retain recent history at settings $k = 0, 2, 6$. The hybrid reference combines runtime-derived records, including failed attempts, with model-written semantic content. Its variants remove failure records, retain recent history, or cross tool versus inline updates with an existing versus task-adapted schema. The reference condition is shared by these hybrid comparisons. The adapted schema was developed using experience from the same task set, so that comparison is exploratory and in-sample.

The parameter k denotes the configured retention setting. The available records do not resolve individual messages versus complete exchanges consistently across implementations. We therefore do not interpret equal values of k as a verified equality of serialized context. The formal model describes information sources and update order; it does not specify hidden reasoning retention or an exact API message layout.

4.2 Outcomes and resources

For a condition a, task i, and repetition r, let Y_{iar} denote binary success. With n tasks and R repetitions, the solved rate is

$$\widehat{s}_a = \frac{1}{nR} \sum_{i=1}^{n} \sum_{r=1}^{R} Y_{iar}. \tag{5}$$

Differences in solved rate are percentage-point differences against the named reference within the same comparison.

Token measures distinguish peak prompt size, the maximum input context of a model call, from cumulative uncached input, cache reads, and output over the execution. The model-written comparison uses the fixed reference tariff

$$D = 10^{-6}(0.15U + 0.03K + 0.50O), \tag{6}$$

where U, K, and O denote uncached input, cache-read, and output tokens. Costs are in US dollars under this convention, not provider invoices. The hybrid comparison supplies reported token and cost medians; its pricing calculation and per-call prompt peaks cannot be reconstructed from the available aggregates.

We report median execution cost and cost per observed success, $\sum D / \sum Y$, over retained executions, including failures. Its reciprocal is successes per dollar. Successes per million tokens use total recorded input, cache-read, and output tokens as the denominator. These metrics describe the retained workload, excluding discarded attempts and unretained retries. They do not estimate the cost of retrying until success.

A second resource comparison is conditional on both policies solving the same task in the same repetition. For each resource, we report the median of within-match condition/reference ratios. The selected subset differs by condition. This estimand describes relative resource use on shared successes, whereas cost per success also reflects failures and differences in capability. A ratio of marginal medians is not substituted for a median of matched ratios.

4.3 Statistical analysis

Solved-rate tables report two-sided exact McNemar p-values for the named matched contrasts. Conditional cost-ratio tables use a two-sided exact sign test against a ratio of one, excluding ties. These are unadjusted, exploratory reference tests under an independence assumption for matched observations. Repeated tasks and possible execution dependencies limit that interpretation. The tests do not adjust for within-task dependence or control the familywise error rate.

For the model-written comparison, pointwise task-bundle bootstrap intervals provide a complementary assessment of solved-rate differences. We resample task identities with all their conditions and repetitions, using 100,000 resamples and seed 20260922. Percentile endpoints define the 95% intervals. This preserves the observed within-task structure under a task-resampling interpretation, without establishing that the benchmark is a random sample. The hybrid aggregates do not retain the task-level covariance needed for the same analysis. Small differences or non-significant tests are not interpreted as equivalence.

The supplementary horizon analysis partitions tasks by the full-history control's median recorded turns, T, at $T = 11$. Membership is fixed across state conditions. This post-hoc stratification describes shorter and longer reference trajectories; it does not measure a causal effect of horizon.

4.4 Analysis material

The model-written results are recomputed from released execution-level records using their canonical inclusion rules. The hybrid results use final condition summaries and matched discordant counts. This distinction sets the resolution of the analyses: the latter support the reported contrasts but not reconstruction of individual trajectories. The reproducibility appendix specifies the available material and derived metrics.

5 Retaining recent interaction history

Replacing full history with model-written state reduces the solved rate from 61.3% to 34.9% (Table 1). Retaining recent history changes the result: the rate reaches 45.2% at $k = 2$ and 59.0% at $k = 6$. The largest setting's difference from full history is -2.3 percentage points.

Table 1: History retention and task success. Each policy family has its own full-history reference. Differences and exact McNemar tests use that reference; the panels are not pooled.

Approach	k	Solved rate (%)	Δ (pp)	p
Model-written memory				
Full history	–	61.3	–	–
Model-written state	0	34.9	-26.4	2.1×10^{-13}
Model-written state	2	45.2	-16.1	9.7×10^{-7}
Model-written state	6	59.0	-2.3	0.488
Hybrid memory				
Full history	–	51.7	–	–
Hybrid state	0	39.7	-12.1	0.001
Hybrid state	2	40.8	-10.9	0.001
Hybrid state	6	48.3	-3.4	0.392

The task-bundle interval for the state-only difference is $[-34.1, -18.8]$ percentage points. For state with $k = 6$, it is $[-7.7, +3.4]$. The latter permits both a practically relevant deficit and a modest improvement, so proximity of the point estimates does not establish equivalence.

Recent history also improves the hybrid policy. Its solved rate rises from 39.7% without retained history to 40.8% at $k = 2$ and 48.3% at $k = 6$. The largest setting improves on hybrid state by +8.6 percentage points, while its full-history reference reaches 51.7%. The agreement in direction across the two families is useful, but the effects describe different implementations and must not be combined into a single estimate.

These comparisons identify retained interaction records as a consequential design choice. Such records can preserve details that a state writer omits, including the context of an observation or the sequence linking an edit to a test outcome. That is a possible explanation, not a measured failure taxonomy. The experiment does not separate loss of information from poor use of preserved information, changes in formatting, or altered stopping behavior.

The result also leaves the role of state itself open. State plus recent history is compared with state alone, not with the same recent-history window without state. A history-only control is needed before attributing an additional benefit to the state block. The supplementary comparison by reference trajectory length appears in Appendix B.

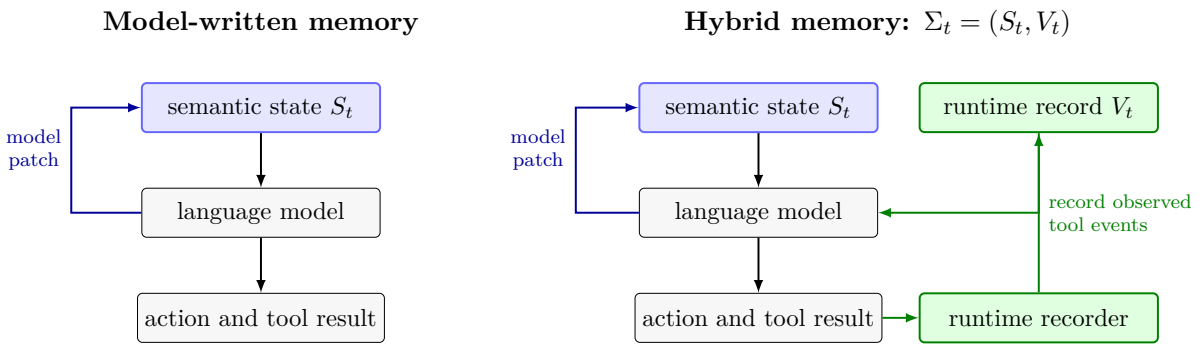
6 Hybrid state and failure records

Hybrid state changes who records execution evidence. Observable outcomes are retained by the runtime, while the model maintains their semantic interpretation and pending work. This division is intended to reduce the need for the model to rewrite facts already available to the system. The question is whether the resulting representation is sufficient when the interaction transcript is removed. Figure 2 shows the division of state ownership.

The hybrid reference reaches a solved rate of 39.7%, against 51.7% for full history (Table 2). The version without failure records reaches 38.5%, a difference of −1.1 percentage points from the hybrid reference. This comparison provides little evidence of a large benefit from the tested failure-record representation, but does not establish that such records are unnecessary.

The success deficit remains even when observable evidence is partly runtime-maintained. Recording an event and selecting the information needed for a later decision are different operations. A command outcome may be accurately retained while its relevance to a new failure remains unclear. The measurements do not distinguish this possibility from other weaknesses of the hybrid implementation.

The runtime comparison also should not be read as a direct estimate of the benefit of adding runtime records to model-written state. Those families have separately evaluated controls, and the hybrid system includes other implementation differences. The matched failure-record



The model updates S; the runtime updates V.
A runtime record preserves captured evidence, not a complete account of the environment.

Figure 2: State ownership. Hybrid memory retains model-written semantic content and adds runtime-derived records. Recording an event does not establish that an extracted interpretation is correct or that all relevant events were captured.

Table 2: Hybrid state with and without failure records. Token use and cost are per-execution medians; token counts are in thousands. Exact McNemar tests compare each policy with full history.

Approach	Solved rate (%)	Output (ktok)	Cached (ktok)	Median cost ($)	p
Full history	51.7	16.4	62.5	0.0104	–
Hybrid, no failure records	38.5	19.1	10.2	0.0127	4.3×10^{-4}
Hybrid state	39.7	17.3	9.3	0.0120	0.001

contrast isolates a narrower change within the hybrid system. Its strongest observed state-based result comes from retaining recent history, as shown in Table 1.

7 Update protocol and state schema

The hybrid comparison crosses the update interface with the semantic schema (Table 3). The tool-based reference with the existing schema reaches 39.7%. Replacing that schema with the task-adapted version raises the observed solved rate to 41.4%, a +1.7 percentage-point change. Inline updates with the existing schema instead reach 35.1%, a −4.6 point change from the reference.

Table 3: Update protocol and schema within hybrid state. Output tokens and cost are per-execution medians. Exact McNemar tests compare each configuration with Tool + Existing schema.

Protocol	Schema	Solved rate (%)	Output (ktok)	Median cost ($)	p
Tool	Existing	39.7	17.3	0.0120	–
Inline	Existing	35.1	28.6	0.0165	0.185
Tool	Task-adapted	41.4	21.7	0.0137	0.678
Inline	Task-adapted	28.7	33.1	0.0190	0.001

Combining inline updates with the adapted schema yields 28.7%, below either change in isolation. The descriptive difference-in-differences for protocol and schema is −8.0 percentage points. This is the difference between the inline effect under the adapted schema and its effect under the existing schema, not the joint configuration's difference from the reference. The available aggregates do not support a task-level interaction test.

Neither intervention recovers the within-comparison full-history rate of 51.7%. The result does not support the hypothesis that these particular protocol or schema changes are sufficient to repair the observed deficit. It also does not rule out better schemas or more faithful implementations of the published runtime.

The inline representation changes both how actions are expressed and when state maintenance occurs. Its larger reported output median may include additional patch content or changes elsewhere in the trajectory. Attributing the success difference to formatting burden, patch validity, or a specific omitted field would require a controlled trace-level analysis. Schema adaptation is likewise an implementation choice here, not an independently validated memory abstraction.

8 Prompt size, token use, and cost

A smaller prompt does not necessarily make successful execution cheaper. For model-written state without retained history, the median peak-prompt ratio is 0.47, but the cost ratio is 2.57 on shared successes (Table 4). The corresponding output-token ratio is 4.55. The compact

representation therefore coexists with higher output use and higher reference-tariff cost in this subset.

Table 4: Resource ratios on shared successes: median of matched condition/full-history ratios. Peak prompt is a per-call maximum; the token columns are cumulative. The exact sign test concerns cost ratios.

Approach	Cost	Peak prompt	Input	Output	Cached	p
Model-written state	2.57×	0.47×	1.86×	4.55×	0.89×	1.4×10^{-12}
State + history ($k = 2$)	1.39×	0.60×	1.62×	2.08×	0.40×	4.7×10^{-4}
State + history ($k = 6$)	1.15×	0.81×	1.96×	1.36×	0.21×	0.036

Retaining history brings the conditional cost ratio toward one: 1.39 at $k = 2$ and 1.15 at $k = 6$. These ratios do not describe the whole workload. They omit both failures and successes that only one condition achieves, precisely the outcomes on which capability differs.

Table 5 includes retained failures and reports cost and successes per dollar. Taking the reciprocal of the latter gives cost per observed success. State without history costs \$0.1630 per success, compared with \$0.1116 for full history. At $k = 6$, the value is \$0.0768, below that reference despite the higher conditional cost ratio. There is no contradiction: the conditional comparison measures resources on shared successes, while the all-execution comparison includes changes in success and termination.

Table 5: Cost and efficiency over retained executions, including failures. Successes per dollar use aggregate cost and observed successes, not ratios of medians. Token efficiency is reported where cumulative totals are available. Cost conventions differ between panels.

Approach	Median cost (\$)	Mean cost (\$)	Solved/\$	Solved/Mtok
Model-written memory				
Full history	0.0137	0.0684	8.96	0.421
Model-written state	0.0239	0.0568	6.14	1.245
State + history ($k = 2$)	0.0151	0.0493	9.16	1.964
State + history ($k = 6$)	0.0154	0.0453	13.02	2.718
Hybrid memory (efficiencies approximate)				
Full history	0.0104	0.0245	21.13	–
Hybrid state	0.0120	0.0186	21.36	–
Hybrid + adapted schema	0.0137	0.0196	21.11	–
Hybrid + inline	0.0165	0.0230	15.25	–
Hybrid + inline + adapted schema	0.0190	0.0290	9.92	–
Hybrid + history ($k = 2$)	0.0122	0.0189	21.65	–
Hybrid + history ($k = 6$)	0.0144	0.0242	19.95	–

Figure 3 places these history-retention policies in the success–cost plane. The panels keep their own controls and accounting conventions, so horizontal positions are compared only within a panel.

Across retained executions, model-written state alone has a lower median peak prompt but higher median output and more recorded turns than full history (Table 6). The largest history-retention setting increases uncached input, while bringing output closer to the full-history level. Cache-read counts differ substantially across policies. These observations establish resource differences, but do not isolate cache invalidation, repeated inspection, or patch generation as their causes.

The economic objective determines which comparison matters. Peak prompt size concerns context capacity; cost per success concerns the retained workload; conditional ratios compare

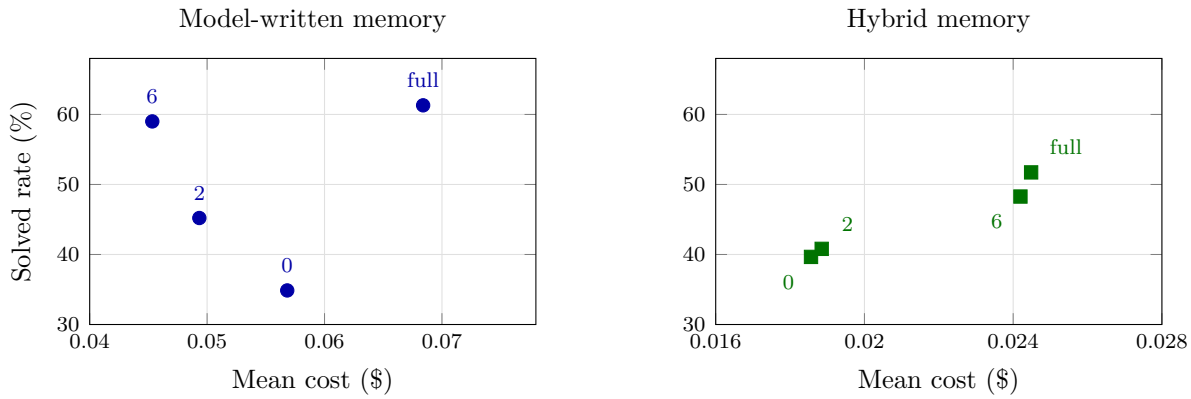


Figure 3: Success–cost trade-offs when retaining history. Labels are the retention setting k; full denotes the full-history reference without a state block. Each panel includes failures in mean cost and uses its own control and cost convention. The horizontal scales differ.

Table 6: Median recorded turns T, per-execution peak prompt, and cumulative token use for model-written state policies. Token counts are in thousands; input excludes cache reads. Peak is the median of execution maxima, not the median per-call prompt. Token columns are separate marginal medians and cannot be summed to obtain median total tokens.

Approach	Median T	Peak prompt (ktok)	Input (ktok)	Output (ktok)	Cached (ktok)
Full history	13	16.1	28.8	16.4	87.4
Model-written state	25	4.0	24.9	37.4	36.7
State + history ($k = 2$)	14	6.2	30.3	20.1	19.1
State + history ($k = 6$)	11	11.3	53.3	16.6	14.8

resources when both policies finish. An evaluation based only on compression can miss a capability deficit, while one based only on aggregate expense can favor cheap unsuccessful executions. Reporting these quantities together makes the trade-off visible without treating any single measure as a universal efficiency score.

9 Threats to validity

Implementation and selection. The findings concern specific implementations, not structured memory in general. Complete execution-source snapshots, generation settings, and prompt traces are unavailable for independent replay. The model-written release specifies inclusion rules, but their prospective timing is not independently established. The hybrid collector description leaves budget-related exclusions ambiguous, so outcome-dependent selection cannot be ruled out. Conditions were not demonstrably balanced over execution time and provider route.

Measurement and inference. Verifier-defined success measures benchmark completion rather than memory quality in isolation. The available retention parameter does not establish identical message semantics across implementations. Reference-tariff cost and reported cost are distinct accounting conventions, and neither includes all discarded attempts. Repeated tasks, outcome-conditioned resource subsets, and exploratory multiple comparisons limit statistical interpretation. The reported intervals do not establish representative task sampling, and the hybrid aggregates do not support the same task-level uncertainty analysis.

Generalization. The evaluation uses one model and coding-agent configuration on a limited task collection. Schema adaptation used that same collection. Results need not transfer to other models, state writers, retrieval mechanisms, or workloads near a context-window limit. The missing history-only control prevents a claim that state and recent history are complementary. No validated trace-level analysis identifies a causal explanation for the observed deficits.

10 Related work

SKILL.state [Badhe et al., 2026] replaces append-only history with mutable task state. Its tests include both synthetic environments and interactive benchmarks. The present work studies transfer to terminal-based coding, with different model, runtime, and benchmark choices. Its negative results concern the tested systems and do not contradict the original results.

An imposed event horizon differs from an agent's chosen trajectory. An externally specified event stream lets researchers measure behavior as the number of events grows. In an interactive coding task, the policy also affects how many actions are taken before the agent stops. Lower context growth at a fixed horizon does not determine the cost or success of such trajectories.

Prompt compression offers another approach to reducing model input. LLMLingua [Jiang et al., 2023] uses budgeted, token-level compression to shorten prompts. Replacing history with state instead requires the agent to maintain selected details as explicit fields. The two approaches differ in what they preserve and how they encode it; this study does not compare them directly.

ReAct [Yao et al., 2023] interleaves reasoning and actions with environmental feedback. A context policy governs which earlier records remain available within such a loop. Our comparisons vary that policy without claiming that retained records alone determine how well the agent reasons. Terminal-Bench [Merrill et al., 2026] provides the executable tasks and verifier outcomes used to measure the resulting task-level behavior.

11 Conclusion

The tested state representations do not preserve the task success of full interaction history when used as its replacement. Retaining recent records raises the solved rate in both model-written and hybrid state policies, while the tested inline protocol and task-adapted schema do not recover the full-history result. These findings identify useful information in the retained interaction record without establishing the incremental benefit of state over an equally sized history window.

Resource savings depend on the objective. Smaller peak prompts coexist with higher conditional successful-execution costs, whereas all-execution cost per success can improve when recent history is retained. State-based context management should therefore be evaluated through task completion, token categories, and cost together. The next discriminating comparison is a matched history-only policy, followed by replication across models and workloads before broader claims about structured memory are warranted.

References

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. *arXiv preprint arXiv:2608.26263*, 2026. URL `https://arxiv.org/abs/2608.26263`. v3, 02 Sep 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*,

2023. doi: 10.18653/v1/2023.emnlp-main.825. URL https://aclanthology.org/2023.emnlp-main.825/.

Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868. arXiv:2601.11868. Version 2.0 used in this work.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

A Reproducibility and metric definitions

The analysis artifact contains execution-level records for the model-written policies and final aggregate summaries for the hybrid policies. Table entries and numerical claims are generated from those inputs. The aggregate summaries contain solved-rate margins and matched discordant outcomes, sufficient to reconstruct the reported success contrasts and exact McNemar tests. They do not contain complete trajectories, per-call prompt measurements, or joint task-level outcomes for all configurations.

Resource ratios are computed separately for each condition against its own full-history reference, restricting to shared successes. For a resource X, the reported statistic is

$$\text{median}_{(i,r):\, Y_{iar}=Y_{i0r}=1} \frac{X_{iar}}{X_{i0r}}.$$

A zero reference denominator would make an individual ratio undefined. The analysis does not replace such a denominator with an arbitrary constant. Input and cached tokens are separate recorded counters.

The efficiency measures use all retained outcomes:

$$\text{Solved/\$} = \frac{\sum Y}{\sum D}, \qquad \text{Solved/Mtok} = \frac{10^6 \sum Y}{\sum (U + K + O)}.$$

The latter measures logical token volume under the recorded counter convention, not equivalence of computational work across cached and uncached tokens. Hybrid cost efficiencies are approximate because the available aggregate costs are rounded. Unavailable token totals and prompt peaks are not reconstructed from marginal medians.

B Success by reference trajectory length

Table 7 separates tasks using the full-history control's median recorded trajectory length. All state policies use the same partition. The larger state-only deficit appears in the longer-reference subset, while recent-history retention improves the observed solved rate in both subsets. Because the partition is post-hoc and trajectory length also reflects task characteristics, this pattern does not identify a causal horizon effect or a context-window crossover.

Table 7: Solved rate by the full-history control's median recorded turns T. Differences and exact McNemar tests use full history within the same subset. The grouping is descriptive and held fixed across conditions.

	Control median $T \leq 11$			Control median $T > 11$		
Approach	Solved rate (%)	Δ (pp)	p	Solved rate (%)	Δ (pp)	p
Full history	56.3	–	–	66.7	–	–
Model-written state	39.3	-17.0	6.6×10^{-5}	30.2	-36.5	1.7×10^{-9}
State + history ($k = 2$)	45.2	-11.1	0.011	45.2	-21.4	4.2×10^{-5}
State + history ($k = 6$)	55.6	-0.7	1.000	62.7	-4.0	0.473